

Asset Pipeline of Fools Engine

Work in progress

Asset Types

Texture

Texture2D

Shader

ShadingModel

Material

Mesh

RenderMesh

Model

Skeleton

SkinnedModel

Animation

Scene

Prefab

Audio

Storage structure

Editor and Game

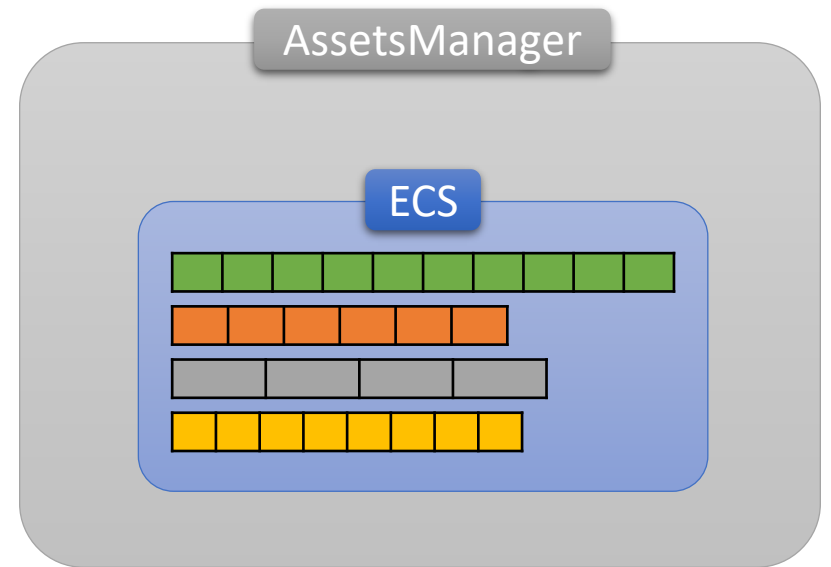
Assets are broken down into components and stored in ECS.
Components use AC prefix (from Asset Component).

Common Components:

- ACAssetType
- ACUUID
- ACFilepath
- ACSourceFilepath
- ACMasterAsset
- ACRefsCounters

Asset does not have to have all of those components (more on that later).

Every asset has a CoreComponent, that is different for every Asset type.
It contains metadata about the asset and a pointer to the data in CPU memory.
CPU side representation of GPU stored copy of the asset is also a component.



Asset Archetypes

Project Asset

- Assets that the game developer operates on.
- Components:
 - ACAssetType
 - ACUUID
 - ACFilepath
 - ACSourceFilepath
 - ACRefsCounters
 - CoreComponent
 - other, asset type specific components

Internal Asset

- Assets often contain references to other assets. Internal assets are only ever referenced by other assets. Fully hidden from the user.
- Components:
 - ACAssetType
 - ACUUID
 - ACMasterAsset (ref back to Project Asset)
 - CoreComponent
 - other, asset type specific components

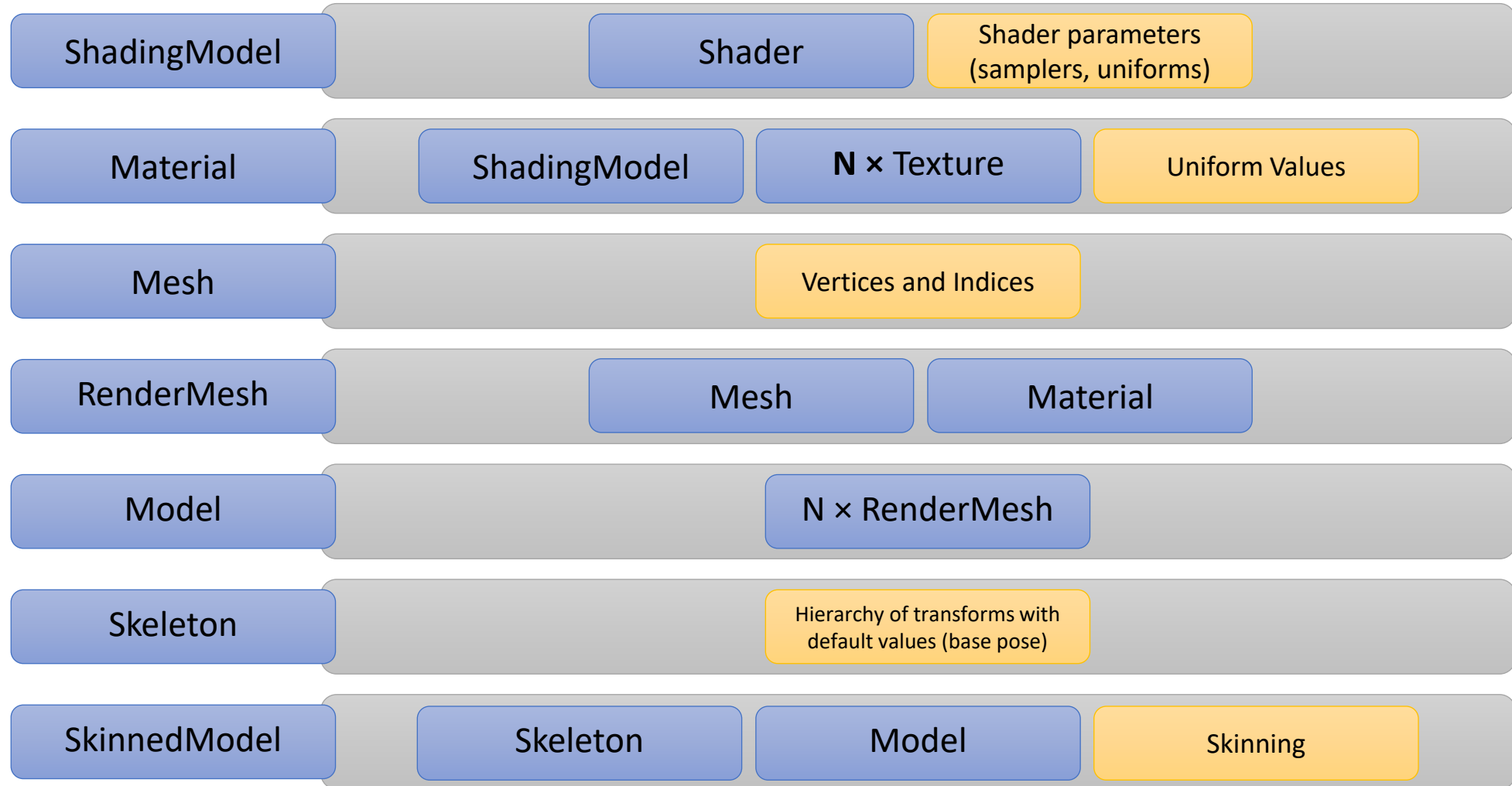
Editor Asset

- Assets used by only by editor (e.g. icons).
- Components
 - ACAssetType
 - CoreComponent
 - other, asset type specific components

Base Asset

- Assets build-in to the engine used by default.
- Components
 - ACAssetType
 - ACUUID
 - ACFilepath
 - CoreComponent
 - other, asset type specific components

Rendering Assets



Scene Components for Geometry Assets

Geometry assets (sufficient for rendering on their own) have dedicated scene components.

CRenderMesh

- AssetHandle<RenderMesh>

CModel

- AssetHandle<Model>

CSkinnedModel

- AssetHandle<SkinnedModel>

They also have scene components “simulating” their existence – asset views. That allows for runtime dynamism.

CRenderMeshView

- AssetHandle<Mesh>
- AssetHandle<Material>

CModelView

- Vector<AssetHandle<RenderMesh>>

CSkinnedModel

- AssetHandle<Skeleton>
- Vector<AssetHandle<RenderMesh>>
- Skinning

Runtime Management

Editor and Game

Centralized streaming:

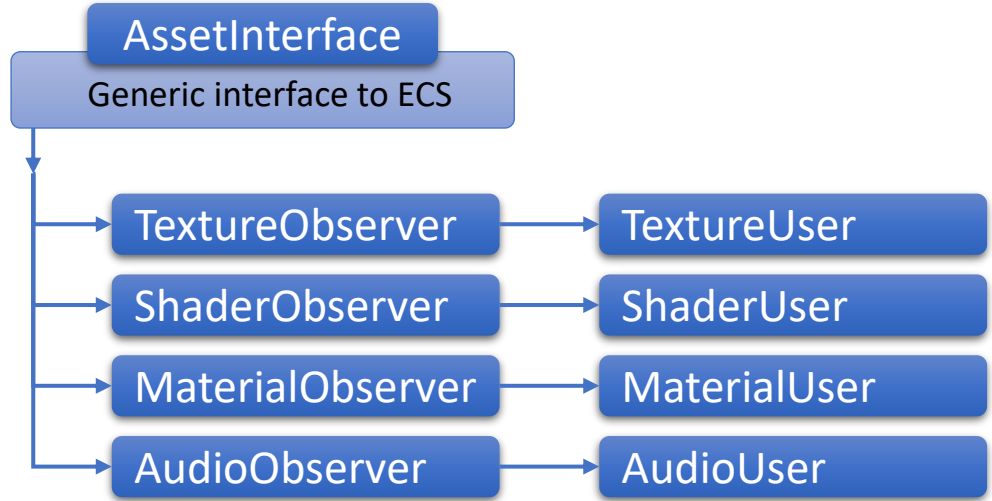
You can never assume an asset is loaded and there is no explicit asset loading request possibility. Instead, a handle to the asset has a loading priority value - enum. Asset handles of each loading priority are atomically counted in a component of that asset (reference counting). Asset Manager uses those counts to decide centrally which assets to load and unload:

1. Calculates score of each asset as $\sum$ of count*priority
2. Sorts assets based on those scores
3. Loads as many assets as much memory is available going from the highest score down (and unloading going from lowest score up)

Asset loading/unloading and score calculation/sorting runs in an asynchronous loop, but in a foreground (structured concurrency). This makes dynamic adaptive asset loading management much easier and safer (e.g. preloading assets for next level simply by setting their handles from None to Low). There is no need for the user to write asynchronous multithreaded code and still get it's benefits.

```
enum AssetLoadingPriority : uint32_t
{
    None = 0,
    Low = 1,
    Standard = 10,
    High = 100,
    Critical = -1
};
```

Access

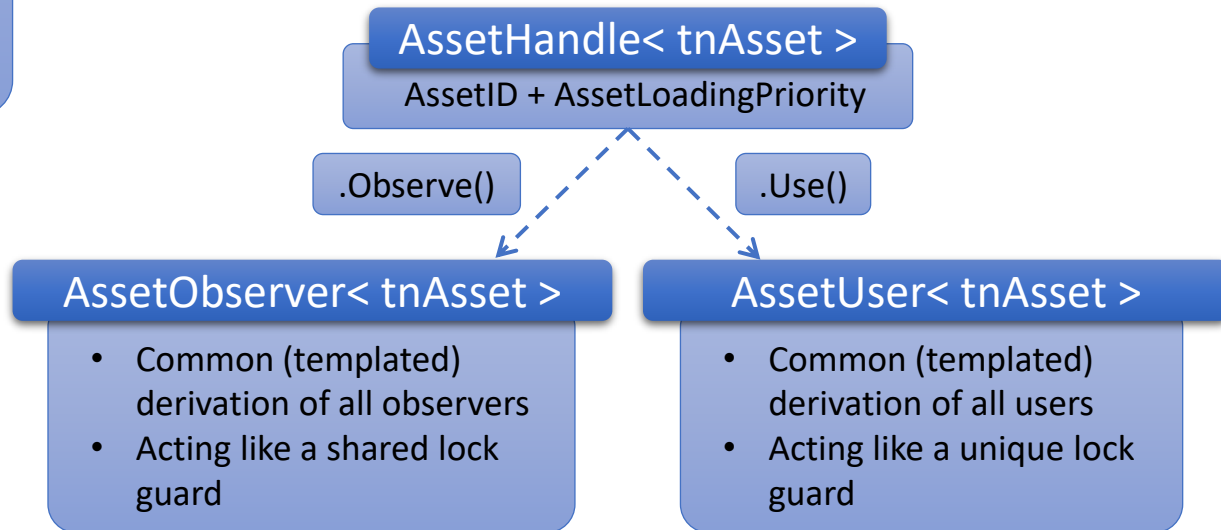


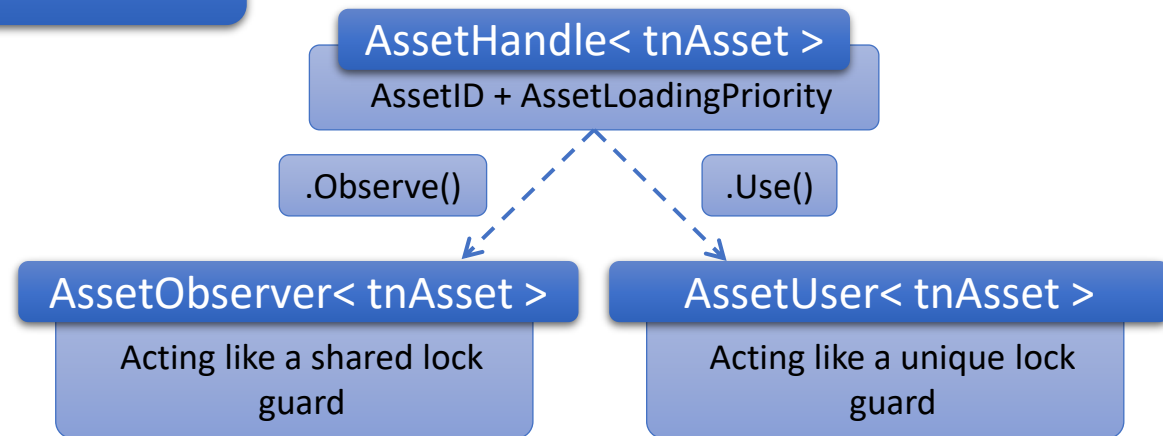
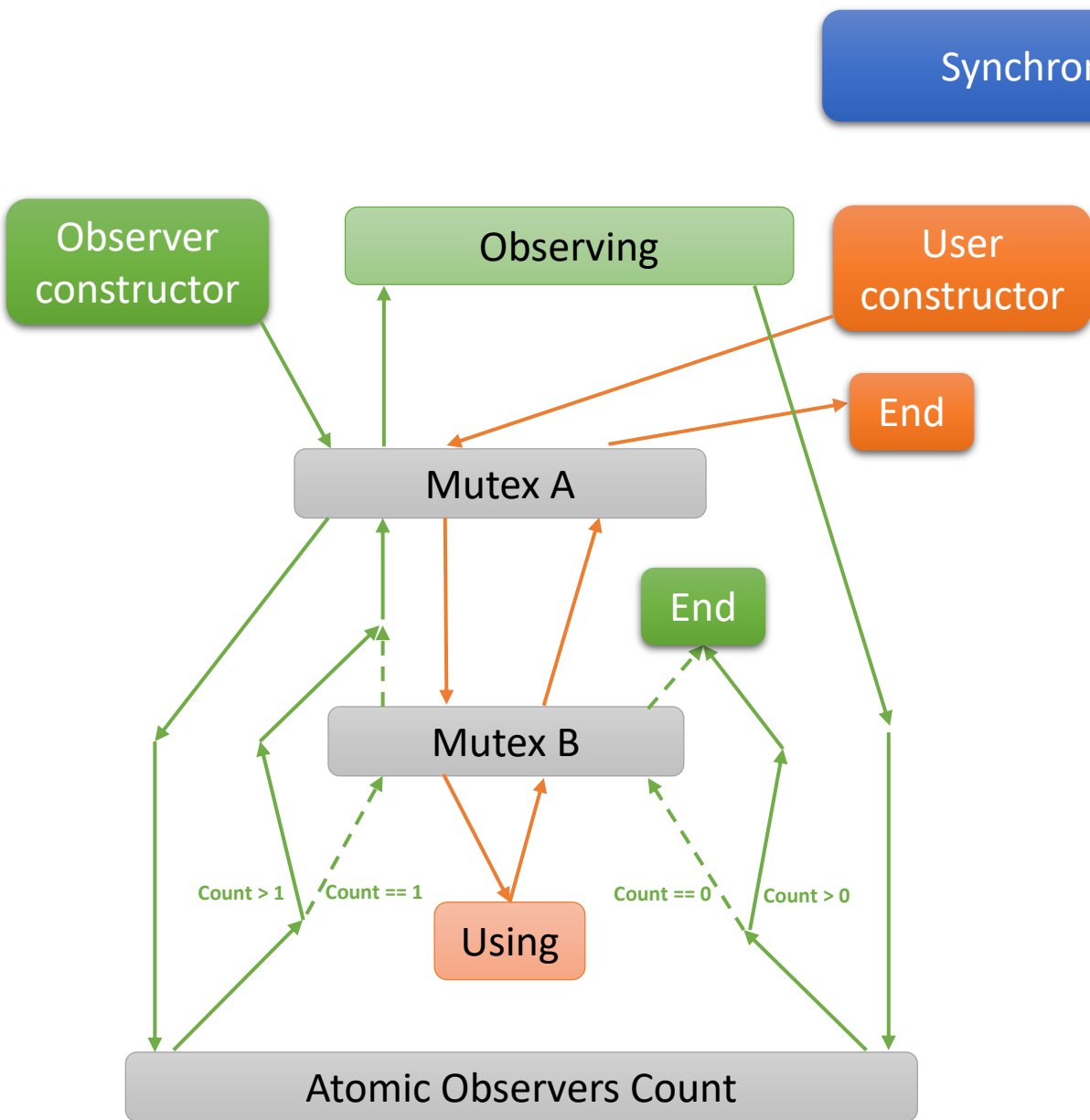
Observer

- Interface to a concrete Asset
- Public only methods specific to its asset type
- Can be conceptualized as an asset itself, but actually is just an ECS handle (Asset ID + Registry ptr)
- Treats Asset as const

User

- Extends Observer adding methods modifying an Asset





Observer() 1. Lock Mutex A 2. Observers Count ++ 3. If 2. was 0->1 • Lock Mutex B 4. Unlock Mutex A	User() 1. Lock Mutex A 2. Lock Mutex B
~Observer() 1. Observers Count -- 2. If 1. was 1->0 • Unlock Mutex B	~User() 1. Unlock Mutex B 2. Unlock Mutex A

Synchronized Runtime Access Examples

```
void Renderer::Draw(
    const Ref<VertexBuffer>& vertexBuffer,
    const AssetObserver<Material>& materialObserver,
    const glm::mat4& transform,
    const glm::mat4& VPMatrix)
{
    FE_PROFILER_FUNC();

    auto& material_core = materialObserver.GetCoreComponent();
    auto sm_observer = AssetObserver<ShadingModel>(material_core.ShadingModelID);
    auto& sm_core = sm_observer.GetCoreComponent();
    auto shaderUser = AssetUser<Shader>(sm_core.ShaderID);

    shaderUser.Bind(s_ActiveGDI);

    shaderUser.UploadUniform(
        s_ActiveGDI,
        Uniform("u_ViewProjection", ShaderData::Type::Mat4),
        (void*)glm::value_ptr(VPMatrix)
    );
    shaderUser.UploadUniform(
        s_ActiveGDI,
        Uniform("u_Transform", ShaderData::Type::Mat4),
        (void*)glm::value_ptr(transform)
    );

    for (const auto& uniform : sm_core.Uniforms)
    {
        auto dataPointer = materialObserver.GetUniformValuePtr(material_core, uniform);
        shaderUser.UploadUniform(s_ActiveGDI, uniform, dataPointer);
    }
}
```

```
auto render_mesh_observer = render_mesh_component.RenderMesh.Observe();

auto& render_mesh_core = render_mesh_observer.GetCoreComponent();
auto material_observer = AssetObserver<Material>(render_mesh_core.MaterialID);
auto shaderID = AssetObserver<ShadingModel>(material_observer.GetCoreComponent().ShadingModelID).GetCoreComponent().ShaderID;
{
    auto shader_observer = AssetObserver<Shader>(shaderID);

    shader_observer.Bind(GDI);
    shader_observer.UploadUniform(GDI, Uniform("u_ViewProjection", ShaderData::Type::Mat4), VPmatrixPtr);
    shader_observer.UploadUniform(GDI, Uniform("u_ModelTransform", ShaderData::Type::Mat4), modelTransformPtr);
    shader_observer.UploadUniform(GDI, Uniform("u_EntityID", ShaderData::Type::UInt), &ID);
}

AssetObserver<Mesh>(render_mesh_core.MeshID).Draw(material_observer);
```